

Vibe Coding Playbook

Production-minded prompts and quality gates for Antigravity, Cursor, and GitHub Copilot without losing control of the architecture.

Recommended build: Option 2 - ASK -> CLARIFY -> DEFINE -> TEST -> LEARN

Source basis: the complete 13-page assignment PDF provided by you. Page references inside these guides refer to that assignment.

1. Vibe-coding workflow for Antigravity, Cursor and GitHub Copilot

Use AI tools as pair programmers, but keep one written project contract so the tools do not gradually add features or change architecture. The company explicitly wants you to understand and explain the submitted code.

Golden rules

- Never ask an AI tool to “build the whole app” in one shot.
- Work in vertical slices: UI shell -> schema -> parse API -> clarification loop -> test -> learn -> polish.
- After each slice, run lint/typecheck/build and manually test the happy path.
- Reject any generated dependency that has no clear job.
- Do not accept generated code you cannot explain.
- Commit at stable checkpoints so you can recover from bad AI edits.

2. Master context prompt - paste at the start of a coding session

You are helping me build a selection assignment called “AI Trading Research Assistant”. The company values critical thinking, ambiguity handling, product thinking, clean architecture and explainability more than feature count or visual effects.

Core flow: ASK -> CLARIFY -> DEFINE -> TEST -> LEARN.

Hard constraints:

- Next.js + TypeScript.
- Server-side AI calls only; no API keys in client code.
- Zod validation for all AI/API structured data.
- Do not build auth, payments, brokerage execution or a full trading dashboard.
- Do not add LangChain/LangGraph/vector DB unless a concrete requirement needs it.
- AI may structure questions and explain evidence; deterministic code owns metrics.
- Every important assumption must be visible and confirmable.
- Distinguish user-stated, assumed, clarified and derived fields.
- Sample/mock data is acceptable and must be labeled honestly.
- UI should be premium, calm B2B/SaaS, responsive and accessible.
- Avoid unnecessary re-renders and unnecessary client components.
- Preserve current architecture unless I explicitly approve a change.

Before editing files: summarize the smallest change, files affected, and acceptance criteria.

After editing: run/recommend typecheck, lint, tests/build and report any remaining issues.

3. Six implementation prompts

Prompt 1 - Scaffold the architecture

Create the minimal project structure for the five-stage research workflow. Do not implement LLM calls yet. Build types, Zod schema, stage reducer/state machine, and static components with realistic mock state. Keep the page responsive. Return only changes required for this slice and explain why each file exists.

Prompt 2 - Premium UI without over-design

Improve only the visual layer. Use a calm Cyprus (#004741) + warm sand palette, clear spacing, strong typography and subtle borders. Keep one primary CTA per stage. Do not add gradients everywhere, 3D, glassmorphism, animations or unrelated dashboard widgets. Ensure keyboard focus, mobile 360px support, loading/empty/error states and accessible labels.

Prompt 3 - Structured AI parser

Implement POST /api/research/parse using a server-only LLM provider.
Input: query + prior clarifications. Output must validate against Zod and contain: experimentDraft, missingFields, assumptions, readyToTest.
Never fabricate market results. Clearly tag field provenance.
Add input length validation, safe errors, output validation and one controlled retry.

Prompt 4 - Clarification merge

Implement a deterministic clarification merge layer. The LLM can propose questions, but merging answers and deciding required-field completeness must be TypeScript code. Do not overwrite user-confirmed values with new AI suggestions. Add tests for this behavior.

Prompt 5 - Deterministic test engine

Implement a transparent sample backtest using bundled/simulated deterministic data. Calculate event count, mean/median forward return, win rate, baseline, edge vs baseline, and apply configured costs/slippage. Explicitly define overlap and entry-timing rules. Do not let the LLM calculate metrics.

Prompt 6 - Learn screen + final hardening

Build the Learn view with two clearly separated sections:
(1) "What the data shows" from deterministic metrics and
(2) "What we can reasonably conclude" from cautious interpretation.
Add limitations and next-research suggestions. Then audit the whole app for security, re-render issues, accessibility, console warnings, error states, unused dependencies, secret leakage and README accuracy. Do not add new features.

4. Anti-re-render checklist for AI-generated React code

- Question any effect that merely derives state from props/state; derive during render instead.
- Question effects that set state immediately on mount without external synchronization.
- Do not put unstable inline objects/functions into effect dependency arrays.
- Keep global workflow state small; do not store derived metrics twice.
- Avoid context providers around the entire app for frequently changing fields if a reducer in the workbook is enough.
- Never use memo/useMemo/useCallback mechanically. Measure or justify.
- Prefer server components outside interactive workflow boundaries.

5. Final pre-submission audit prompt

Act as a strict technical evaluator for this assignment. Do not add features.
Review the repository against these criteria: critical thinking, independent thinking, problem solving, product thinking, technical implementation, communication.
Find: hidden assumptions, confusing UX, unsafe API handling, AI output trust bugs, missing validation, re-render/hydration issues, misleading result language, unnecessary dependencies, code I may struggle to explain, and mismatches with README.
Return findings ranked: BLOCKER / HIGH / MEDIUM / LOW, with minimal fixes.

6. README structure

- Project summary and the research journey.
- Why this problem is ambiguous.

- Architecture diagram.
- Technology choices and why.
- Experiment schema/provenance model.
- How clarification works.
- How testing works and what data is used.
- Known limitations and research risks.
- Security/validation decisions.
- How to run locally.
- Environment variables without secret values.
- AI tools used and what you personally decided.
- What you would improve with more time.

7. 2-3 minute demo script

Time	What to show / say
0:00-0:20	Problem: vague market questions are not directly testable; the product makes ambiguity visible.
0:20-0:50	ASK: enter "Does buying NIFTY after a sharp fall work?"
0:50-1:20	CLARIFY: show missing threshold/holding/test period and confirm them.
1:20-1:45	DEFINE: show structured experiment + provenance/assumptions.
1:45-2:15	TEST: run deterministic sample test and briefly explain metrics.
2:15-2:40	LEARN: evidence vs conclusion, limitations, next questions.
2:40-3:00	Architecture + AI usage + one improvement you would make next.

Best closing line for the demo: "I intentionally kept the prototype small. The main design choice is that the AI can help structure ambiguity, but it cannot silently invent important parameters or fabricate evidence."

Confirmed Implementation Stack

This page supersedes any earlier wording that presented the AI provider or database as optional choices. For this submission, the selected implementation is fixed:

Layer	Selected technology	How it is used
AI / LLM	Google Gemini API	Parse natural-language research questions, identify ambiguity, propose clarification questions
Database	Neon PostgreSQL	Persist research sessions, structured experiments, clarification answers, test runs, and con
ORM	Drizzle ORM	Typed schema, migrations, and database access from the Next.js server layer.
Validation	Zod	Validate both user input and Gemini structured output before persistence or testing.
Frontend / Backend	Next.js + TypeScript	App Router UI plus server-side route handlers/server functions.

Environment variables

```
GEMINI_API_KEY=...  
DATABASE_URL=postgresql://...  
GEMINI_MODEL=gemini-2.5-flash
```

Recommended data flow

Question -> server validation -> Gemini structured interpretation -> Zod validation -> clarification/confirmation -> experiment persisted to Neon -> deterministic TypeScript test -> result persisted -> Learn view.

Security rules

Never expose GEMINI_API_KEY or DATABASE_URL in client code. Keep both outside NEXT_PUBLIC variables. Validate request sizes, parse Gemini output against a strict schema, use parameterized ORM queries, add API rate limiting, and never let the LLM fabricate backtest metrics.

Suggested Neon schema

research_sessions: id, original_question, status, created_at
experiments: id, session_id, instrument, timeframe, entry_condition, exit_condition, holding_period, test_period, filters_json, costs_json, hypothesis, provenance_json
test_runs: id, experiment_id, dataset_source, metrics_json, warnings_json, created_at

Why this choice is strong for the assignment: Gemini is used for language understanding and ambiguity handling, while Neon stores reproducible research state. Numerical evidence remains deterministic and auditable instead of being invented by the model.